STANSE: Bug-finding Framework for C Programs

Jan Obdržálek, Jiří Slabý, Jan Strejček and Marek Trtík

Faculty of Informatics, Masaryk University, Brno, Czech Republic
{obdrzalek,slaby,xstrejc,trtik}@fi.muni.cz

Abstract. STANSE is a modular framework, available free under the GPLv2 license, for finding bugs in C programs using static analysis. Its two main design goals are applicability on large software projects like Linux kernel and extensibility with new bug-finding techniques with a minimal effort. There are currently three bug-finding algorithms implemented within STANSE: AUTOMATONCHECKER checks properties described in an automata-based formalism, THREADCHECKER detects deadlocks among multiple threads, and REACHABILITYCHECKER looks for unreachable code. STANSE has been tested on the Linux kernel, where it has found dozens of previously undiscovered bugs.

1 Introduction

Bug-finding based on static analysis have finally come of age in the last decade. One of the papers to really stir interest was [**?**], showing that static analysis can efficiently find many interesting bugs in a real-world code, as demonstrated on the Linux kernel. This work eventually led to a successful commercial tool called COVERITY [**?**]. Over the years, several other successful tools, like CODESONAR [**?**] or KLOCWORK [**?**], appeared. Unmatched citation

However, such fully-featured tools are neither free to obtain, nor is their code available (e.g. for developing new algorithms or tailoring the existing tools to specific tasks). The existing free tools are usually severally limited in what they can do (e.g. UNO [**?**], SPARSE [**?**], SMATCH). One notable exception is FINDBUGS [**?**,**?**], a successful tool working on Java code. Missing citation

STANSE is intended to fill this gap for C language. STANSE can be viewed in following two general ways:

1. STANSE *framework*: a robust Java library for implementing diverse static analysis algorithms. An implemented algorithm can be immediately evaluated on large real-world software projects written in C as the framework is capable to process such projects. For example, it can process the whole Linux kernel. An implementation of such an algorithm within the framework is called *checker*.
2. STANSE *tool*: a Java application, which integrates STANSE framework and provides a user user friendly way, how to apply the framework. It comprise command line options, GUI, and tracing error reports.

2 Tool Usage

Here we describe an usage of tool Stanse. We focus at a pipeline an user typically follows to analyse a source code. Each subsection describes one step of the pipeline.

2.1 Selecting Source Files

There are several ways to tell Stanse what source files to analyse. Besides the standard possibilities, comprising a single given file, all files from a given directory, all files listed in a given file, Stanse can also derive all the necessary information from a project Makefile. In the C language family case , it also remembers compiler flags for preprocessing purposes. This functionality was inspired by the Sparse [**?**] tool. Stanse can currently process source files written in C language fulfilling the ISO/ANSI C99 standard and possibly including almost all of the GNU C extensions.

Why is there: "In the C language family case"

2.2 Specifying checkers

A static analysis algorithm integrated into Stanse is called a *checker*. An user can choose, which checkers should be applied to selected source files. This is very simple procedure for the user, since it only consists of choosing appropriate checkers from a list of checkers, plus choose for each checker, whether interprocedural or intraprocedural variant of a checker should be used.

2.3 Executing checkers

When an user is done specifying source files and checkers to be applied, the verification can start. For an user it means to *push a button*. The verification is then completely autonomous, it does not require any user's assistance. The user can just watch the progress of the execution in the output window. When the execution is finished, a complete list of *error reports*, found by checkers, is showed to the user.

2.4 Processing Error Reports

An error report represent a path in the source code leading to an error. For each source code line along the path of the report there is an information attached explaining what is going on at the line. Tool Stanse provides several possibilities to present the error reports: they can be printed to the console, displayed using a built-in GUI browser, or saved to an external file. In the GUI browser an user can trace each report per line of the source files and read the attached information. Since some of the reports may be false alarms, the user can mark reports in the browser as real errors or false alarm to distinguish them. The browser window is shown in Figure 1.

There are not marked errors and false alarms in the Figure 1!

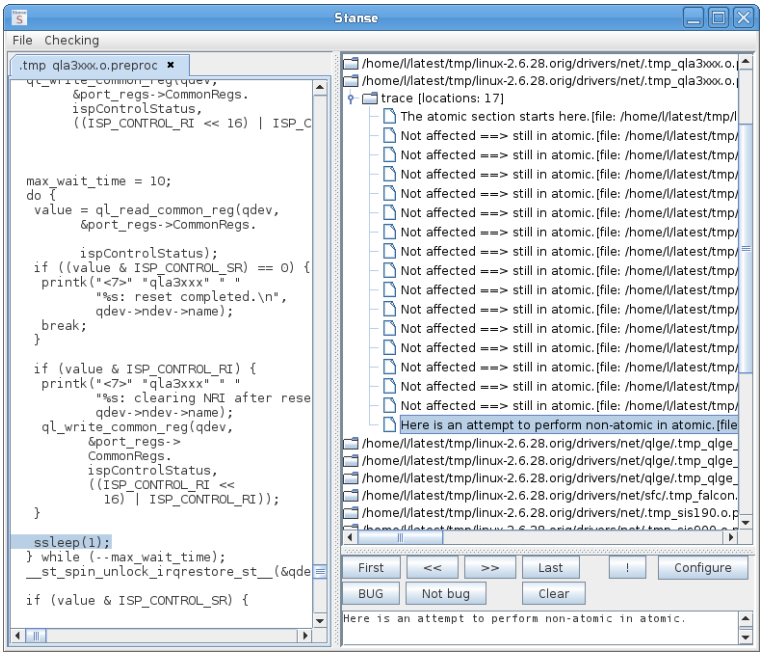

Fig. 1. The GUI browser of error reports.

3 Stanse Framework

Stanse framework is a Java library which is responsible for verification itself. When the framework is passed filled *Configuration*, which means list of source files and list of checkers to be applied, it handles files parsing and executing checkers and collecting error reports. The list of error report is an output from the library. In the following we describe main concepts and properties of the framework's functionality.

3.1 Concept of Checkers

A *checker* is an implementation of a static analysis technique. There can be many checkers simultaneously running in Stanse, but they do not see each other. They run independently. The checkers share read-only lazy representation of checked program and they are gained an access to the libraries providing common types of navigation in a code.

The checkers are integrated to the framework of Stanse using concrete factory design pattern. So to insert a new checker to the framework of Stanse one only needs to (a) implement general checker interface, and (b) register the checker to the checkers' factory of the framework. Then the checker has full access to the features of the Stanse and is immediately ready for checking large software projects, like Linux kernel.

3.2 LazyInternalStructures

The code to be verified is parsed into set of ASTs, CFGs, and call-graphs, representing *internal structures*. These structures can be accessed by checkers through common interface called *LazyInternalStructures*. Since the interface provides only read access, the checkers share the internal structures. Checkers may attach their data to internal structures using for example hash tables, etc. In addition, the interface provides interprocedural or intraprocedural access to the internal structures, depending on the variant of the checker.

There is another important property of the interface LazyInternalStructures. Besides providing common access to internal structures it also enables *streaming*.

3.3 Streaming

As mentioned earlier, the design of the framework focus at large software projects. Such projects consist of hundreds or thousands of modules (compilation units). In practice, it is often impossible to store all the corresponding internal structures in the memory. The Stanse framework therefore applies automatic *streaming* of the internal structures, currently on a module basis.

Instead of parsing all source modules at the beginning, a module is streamed in only when Stanse needs to access some internal structures corresponding to the module. In other words, the internal structures are constructed in a lazy manner – we call them *lazy internals*. If the memory occupied by internal structures exceeds a given limit, some internal structures have to be freed before another module is streamed in. The implemented least-recently-used algorithm discards the structures of a module if they are not accessed for a significant time. If the discarded structure is later accessed again, the corresponding module is streamed back in. The streaming is completely transparent to checkers. It means that checkers may assume all the modules are loaded in the memory all the time. It is framework's responsibility to optimally plan streaming of modules in the memory to provide checkers the illusion that all the modules are in the memory.

3.4 Traversing the Parsed Code

Although all the internal structures are ready to be fully used by a checker at this point, the checker would have to walk through the structures on its own. To prevent unnecessary reimplementations, the most important and heavily used traversal methods are implemented directly inside the framework. A checker can simply use this functionality by specifying

- whether it should go through the structures forwards or backwards, breath-first or depth-first,
- if the interprocedural walk-through should be performed or not, and
- derive a visitor object whose callback method `visit()` will be called for each visited node in CFG.

3.5 Statistics of Error Reports

One may need to know some statistical information about error reports obtained from the verification. The framework contains library, which provides such computations. An user may compute information like how many error reports were received from a source file, how many percent of reports were generated from a specific checker, percentage of some type of error report to all reports, how many reports were already sorted to real errors and false alarms, etc.

3.6 Quick IDE integration

Since the framework is just Java library, it immediately gives the developers an opportunity to reuse the Stanse framework for quickly implement plug-in for some IDE.

4 Checkers

In this section we describe the three currently available checkers.

AutomatonChecker is heavily influenced by [**?**]. It takes, as an input, a set of finite-state automata that describe some property which we want to check. Properties like locking discipline, interrupt management, null pointer dereference, dangling pointers and many others can be handled this way.

TADY!!!

Compared to the implementation described in [**?**] and [**?**], the technique used differs in several aspects. Here is a list of the most important differences: (1) we do not use metacompilation, (2) automata are not input-language specific, a pattern matching is used instead, (3) our analysis is path-sensitive on checker level, but for this reason (4) the interprocedural analysis is done in the context of a single input file.

The patterns in the configuration are specified in the same language as AST is represented, so currently in XML. The idea behind was to isolate the patterns from the source code.

ThreadChecker aims to check for possible deadlocks in concurrent programs. The technique used is based on the notions of locksets of [**?**] and deadlock detection by looking for cycles in resource allocation graphs (RAG). ThreadChecker first tries to identify the parts of the code which can run in parallel, as different threads, then it builds a set of *dependency graphs* for each such thread. A dependency graph statically represents the possible locksets during one execution of a thread. Dependency graphs are then combined and all cycles examined, by building the corresponding RAGs.

```
static void untag_chunk(struct node *p) {
 spin_lock(&entry->lock);
 ...
 new = alloc_chunk(size);
 ...
 spin_unlock(&entry->lock);
}
static struct audit_chunk *alloc_chunk(int count) {
 ...
 chunk = kzalloc(size, GFP_KERNEL);
```

Fig. 2. Bug found in the audit code

ReachabilityChecker searches the internals, a CFG more concretely, for unreachable nodes. These are then reported as warnings or errors, depending on importance (e.g. superfluous semicolons are less important than unused branch). This is an easy task and so the writing of such code should be. The code of the checker is below 200 lines including importance decision and much of strings and comments. So it tries to fully demonstrate the power of the Stanse framework and its primary goal is exactly that. The tool is thus very easy to extend, the checker implementors can focus on real techniques.

explicitne zminit, ze to pouziva traversovani z frameworku.

5 Results – Linux Kernel

We are exercising the Linux kernel by Stanse since 2.6.28 was released back in 2008. It was the first release we decided to test the usability of Stanse on. One reason to choose the Linux kernel is that it represents a large and freely available codebase, where the absence of bugs is of great concern. Moreover, the Linux kernel fully exercises most features of ANSI C99 and GNU C extensions. The other motivation is to provide a free tool for Linux kernel developers.

Currently, Coverity Prevent [**?**] is daily run on the current development branch of the Linux kernel, and the results are reported back to developers. However, developers kept asking for a tool with which their code will be checked before it appears upstream. Moreover, the kernel code is very specific and the tool should consider these specifics.

We used all three checkers described above. These checkers are not kernel specific and only the configuration for AutomatonChecker has to be tailor-made to look for the type of bugs commonly found in the kernel code. The properties checked by the AutomatonChecker in the kernel are:

- correct pairing of functions like balanced locking, check for correct reference counting,
- memory/pointer related bugs like dereferencing null pointers and
- atomic section restrictions to find if someone can cause deadlocks by sleeping inside spinlocks or interrupt handlers.

With this configuration, the running time of Stanse on a common desktop machine with 2 cores and 4 GiB of memory is under 100 minutes. With streaming,

Automaton	Errors	Assorted	Bugs	FP	Ratio
Pairing	266	58	65	143	31.3 %
Memory	86	1	48	37	56.5 %
Atomic	35	1	16	18	47.1 %
Overall	387	60	129	198	39.4 %

Table 1. Errors found by AutomatonChecker in the 2.6.28 Linux kernel. Assorted = Neroztrizene = nepodarilo se rozhodnout.

the memory usage of the Java process oscillates between 400 and 1000 MiB in case of forced garbage collector executions. The standard Stanse Makefile support was used and we did not have to manually tweak the source code in any way in order to be processed by Stanse.

In the 2.6.28 release, more than 70 bugs[1] were reported to kernel developers, most with appropriate patches, and fixed in the next kernel release. Some of these bugs remained undiscovered for seven years[2]. This is despite the kernel being daily checked[3] with Coverity Prevent and we believe it is due to the filters they use.

Now we present one bug which we found in the code reporting security incidents. An excerpt from the particular code is in the Figure 2. `untag_chunk` functions creates a critical section by a spinlock and calls `alloc_chunk` which in turn can sleep in `kzalloc`. This can, under certain circumstances, lead to deadlock of the whole system. The bug was confirmed and fixed.

The false positive (FP) rate oscillates among the different bug types, the overall ratio is that 40 % of all reported errors are real bugs. This result is quite low, however similarly to [**?**], it is without any thorough false-positive filtering techniques (cf. Future Work section). The more detailed results about the AutomatonChecker are in the Table 1, one row per each configuration.

The current kernel versions are checked with other tools like Coccinelle or Smatch. Mostly these tools find simple pattern matching style properties like lock-unlock and are successful at that. We continue to check the kernel releases with Stanse too and, thanks to interprocedural analysis, we still are able to find errors on daily basis.

The most convenient advantage on these tools is their availability. The developers or testers can take them whenever they want and check their code instantly, because they are freely available and they can handle the kernel code. Further these tools need near no configuration (either because they provide predefined one or there is nothing to configure) and are ready to use out of the box.

6 Conclusion

[1] A list of confirmed bugs is available at `http://stanse.fi.muni.cz/bugs.html`

[2] `http://lkml.org/lkml/2009/3/11/380`

[3] `http://marc.info/?l=linux-kernel&m=125856033627134&w=2`

Acknowledgements Jan Kučera is the author of the ThreadChecker. We would like to thank Linux kernel developers, and Cyrill Gorcunov for Stanse alpha testing and useful suggestions. All authors are supported by the research centre Institute for Theoretical Computer Science (ITI), project No. 1M0545.